# Oracle Federation: Combining Multiple Verification Backends

Halley Young
*Microsoft Research*
halleyyoung@microsoft.com

March 23, 2026

**Abstract**

Modern program verification rarely succeeds with a single oracle. An SMT solver may time out on non-linear arithmetic; a runtime witness may be unavailable at analysis time; a human annotation may cover only a fraction of obligations. *Oracle federation* is the principled combination of evidence from heterogeneous backends — SMT solvers, runtime witnesses, AI copilot suggestions, and human annotations — into a single trusted verdict.

We define the oracle model (§2), the federation protocol (§3), and trust-level conflict resolution via a conservative meet in the trust lattice (§4). The *Copilot Fallback Policy* (§5) governs when and how AI-generated evidence may enter the federated verdict. The `SolverAdapter` protocol and `BackendDescriptor` abstraction (§6) decouple the protocol from backend specifics.

Our central result, the *Federation Soundness Theorem* (§7), states that the trust level of a federated verdict equals the meet of all contributing oracle trust levels: trust can never be silently promoted by routing evidence through a high-trust oracle. An implementation in Python (the `jugeo.solver` and `jugeo.foundations.oracle_federation` packages) reduces uncovered obligations on our benchmark suite (90 site calls, 76.7% success rate) while adding sub-millisecond overhead (0.0001 s) per verdict. A machine-verified proof in Lean 4 accompanies this paper; it contains zero `sorry` instances.

# 1 Introduction

## 1.1 The coverage gap in single-oracle verification

The Judgment Geometry framework [**?**] assigns every verification claim a *judgment* 8-tuple $\mathcal{J} = (\mathsf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ where $\mathcal{T} \in \mathbf{T}$ quantifies the epistemic confidence in the evidence bundle $\mathcal{E}$. A judgment is *discharged* when its residual obligations $\mathcal{O}$ are empty; it is *sound* when the trust annotation faithfully reflects the quality of evidence.

A foundational challenge is *coverage*: no single verification backend can discharge all obligations in a real codebase.

- **SMT solvers** (Z3 [**?**], CVC5 [**?**]) excel at quantifier-free arithmetic and bit-vector constraints but time out on non-linear arithmetic, string theories, or large state spaces.
- **Runtime witnesses** provide empirical evidence but are limited to reachable paths and dynamic type information.
- **Human annotations** are authoritative but sparse; requiring full human sign-off on every obligation is impractical.
- **AI copilot suggestions** can fill gaps where other oracles are absent, but their epistemic status is inherently lower than solver proofs.

Oracle federation addresses the coverage gap by *coordinating* multiple backends: each backend attempts obligations within its competence; the results are merged into a single verdict with a trust level that accurately reflects the weakest contributing source.

## 1.2 The silent-promotion hazard

A naive federation strategy — take the highest trust level reported by any oracle — is unsound. If a copilot oracle reports a claim at the COPILOT tier and a solver oracle later confirms the same claim at the SOLVER tier, the SOLVER certificate is legitimate. But if only the copilot responds, the verdict must remain at COPILOT, not SOLVER. We call the erroneous elevation of copilot-tier evidence to solver-tier *silent trust promotion*; our Federation Soundness Theorem rules it out by construction.

## 1.3 Contributions

(1) A formal oracle model with trust ceilings and jurisdiction (§2), and a federation protocol over oracle sets (§3).

(2) A trust-algebra conflict-resolution strategy based on the conservative meet $\sqcap$ (§4).

(3) The `CopilotFallbackPolicy` formalisation, specifying the conditions under which AI evidence enters the verdict (§5).

(4) The `BackendDescriptor` / `SolverAdapter` abstraction that makes federation backend-agnostic (§6).

(5) The Federation Soundness Theorem with a Lean 4 proof (§7).

(6) Experiments on 300 obligations across five backend configurations (§8).

### 1.4 Related work

Nelson and Oppen [**?**] studied the combination of decision procedures for disjoint theories; our federation model is orthogonal — we combine *oracles* (evidence sources) rather than *theories* (logical languages). The architecture is related to portfolio solvers [**?**] but enriched with trust accounting. Trust management in federated security systems [**?**] inspires our trust-ceiling axioms. The judgment 8-tuple and trust algebra are developed in earlier papers of this series [**?**, **?**]; §4's conflict-resolution rules extend the trust meet defined in [**?**].

## 2 Oracle Model

### 2.1 Trust tiers

We use the eight-tier trust lattice $(\mathbf{T}, \preceq)$ of the JuGeo series, recalled here for self-containment:

$$\mathsf{CONTRADICTED} \preceq \mathsf{UNVERIFIED} \preceq \mathsf{COPILOT} \preceq \mathsf{ORACLE} \preceq \mathsf{HUMAN} \preceq \mathsf{RUNTIME} \preceq \mathsf{SOLVER} \preceq \mathsf{PROOF}$$

The meet $\sqcap$ is the greatest lower bound in $(\mathbf{T}, \preceq)$; the join $\oplus$ is the least upper bound. Trust arithmetic is developed in full in [**?**].

**Definition 2.1** (Trust ceiling)**.** A *trust ceiling* $\tau^{\max} \in \mathbf{T}$ is an upper bound on the trust level that an evidence source may assign to its outputs. Formally, if evidence item $\varepsilon$ is produced by a source with ceiling $\tau^{\max}$, then $\tau(\varepsilon) \preceq \tau^{\max}$.

### 2.2 Oracles

**Definition 2.2** (Oracle)**.** An *oracle* is a triple $\mathcal{O}_i = (id_i,\ \kappa_i,\ \tau_i^{\max})$ where:

- $id_i \in \mathbb{S}$ is a unique string identifier;
- $\kappa_i \in \mathsf{BK} = \{\texttt{Z3}, \texttt{RT}, \texttt{ORC}, \texttt{CPL}, \texttt{PRV}, \texttt{HUM}\}$ is the *backend kind* classifying the oracle's evidence generation mechanism;
- $\tau_i^{\max} \in \mathbf{T}$ is the oracle's trust ceiling (Definition **??**).

**Definition 2.3** (Evidence item)**.** An *evidence item* is a quadruple $\varepsilon = (src,\ \phi,\ \tau,\ \Pi)$ where $src \in \mathbb{S}$ is the producing oracle's identifier, $\phi$ is the verified claim, $\tau \in \mathbf{T}$ is the assigned trust level, and $\Pi$ is provenance metadata.

**Definition 2.4** (Ceiling enforcement)**.** Given oracle $\mathcal{O}_i$ and evidence item $\varepsilon$ with claimed trust $\tau$, the *ceiling-enforced item* is

$$\lfloor \varepsilon,\, \mathcal{O}_i \rfloor_\tau \;=\; \big(src,\ \phi,\ \tau \sqcap \tau_i^{\max},\ \Pi\big).$$

**Proposition 2.5** (Ceiling enforcement is sound and idempotent)**.** *For any oracle $\mathcal{O}_i$ and evidence item $\varepsilon$:*

*(a)* $\tau(\lfloor \varepsilon, \mathcal{O}_i \rfloor_\tau) \preceq \tau_i^{\max}$*; and*

*(b)* $\lfloor \lfloor \varepsilon, \mathcal{O}_i \rfloor_\tau,\, \mathcal{O}_i \rfloor_\tau = \lfloor \varepsilon,\, \mathcal{O}_i \rfloor_\tau$.

*Proof.* (a) $\tau \sqcap \tau_i^{\max} \preceq \tau_i^{\max}$ by the meet axiom. (b) $(\tau \sqcap \tau_i^{\max}) \sqcap \tau_i^{\max} = \tau \sqcap (\tau_i^{\max} \sqcap \tau_i^{\max}) = \tau \sqcap \tau_i^{\max}$ by idempotence of $\sqcap$. □

### 2.3 Oracle jurisdiction

An oracle may declare a *jurisdiction* — the set of verification domains it is competent to handle. Routing a query to an oracle outside its jurisdiction is wasteful and can produce low-quality evidence.

**Definition 2.6** (Jurisdiction)**.** An oracle $\mathcal{O}_i$ has *jurisdiction* over domain $d$ if $d \in \mathit{jur}(\mathcal{O}_i)$, where $\mathit{jur}$ maps each oracle to a subset of the global domain set $\mathcal{D}$.

In the implementation, jurisdiction is encoded as a `frozenset` of `VerificationDomain` enum members in `BackendDescriptor.jurisdiction` (§6).

## 3 Federation Protocol

### 3.1 Overview

The federation protocol coordinates a finite set of oracles $\mathfrak{O} = \{\mathcal{O}_1, \ldots, \mathcal{O}_n\}$ to discharge a single obligation $\phi$. Each oracle produces zero or one evidence items; items are ceiling-enforced; the federated evidence is the collection of all non-empty responses with the meet of their trust levels as the federated trust annotation.

**Definition 3.1** (Oracle response)**.** The *response* of oracle $\mathcal{O}_i$ to obligation $\phi$ is

$$\mathsf{Resp}(\mathcal{O}_i, \phi) \;\in\; \{\,\bot\,\} \;\cup\; \{\varepsilon \mid \mathit{src}(\varepsilon) = \mathit{id}_i\,\},$$

where $\bot$ denotes "no evidence produced" (timeout, out of jurisdiction, or internal failure).

**Definition 3.2** (Contributing set)**.** For oracle set $\mathfrak{O}$ and obligation $\phi$, the *contributing set* is

$$S(\mathfrak{O}, \phi) \;=\; \big\{\, \mathcal{O}_i \in \mathfrak{O} \;\big|\; \mathsf{Resp}(\mathcal{O}_i, \phi) \neq \bot \,\big\}.$$

**Definition 3.3** (Federated evidence)**.** Given contributing set $S = S(\mathfrak{O}, \phi) \neq \emptyset$, the *federated evidence* for $\phi$ is

$$\mathsf{Fed}(\mathfrak{O}, \phi) \;=\; \Big(\big\{\, \lfloor \mathsf{Resp}(\mathcal{O}_i, \phi),\, \mathcal{O}_i \rfloor_\tau \mid \mathcal{O}_i \in S \big\},\; \tau_{\mathsf{Fed}}\Big),$$

where the *federated trust level* is

$$\tau_{\mathsf{Fed}} \;=\; \bigsqcap_{\mathcal{O}_i \in S} \tau(\lfloor \mathsf{Resp}(\mathcal{O}_i, \phi), \mathcal{O}_i \rfloor_\tau).$$

If $S = \emptyset$ then $\mathsf{Fed}(\mathfrak{O}, \phi) = \bot$ (no evidence).

### 3.2 Sequential versus parallel dispatch

The protocol admits two dispatch strategies:

**Sequential (priority-ordered).** Oracles are queried in decreasing order of trust ceiling. The first non-$\bot$ response is returned immediately; remaining oracles are not queried. Optimal latency; sacrifices completeness.

**Parallel (broadcast).** All oracles are queried concurrently. All non-$\bot$ responses contribute to the federated evidence. Optimal coverage; higher latency and cost.

The `SolverRouter` (§6) supports both strategies via the `RoutingStrategyKind` enumeration and selects between them based on latency budgets and available oracle jurisdictions.

### 3.3 Compositional obligation splitting

When $\phi = \phi_1 \wedge \phi_2$ the protocol may split the obligation: route $\phi_1$ to one oracle and $\phi_2$ to another, then federate the results. The federated trust is still the meet of both sub-verdicts, so soundness is preserved. The `obligation_splitting` module implements this via a `SplitStrategy` that respects oracle jurisdiction.

## 4 Conflict Resolution

### 4.1 When oracles disagree

Two oracles $\mathcal{O}_i$ and $\mathcal{O}_j$ *conflict* on $\phi$ when one asserts $\phi$ holds (returning evidence) and the other asserts $\phi$ does not hold (returning a counter-model or negative certificate). Conflicts arise from:

- **Incompleteness**: a solver times out and returns $\perp$, while a runtime witness confirms $\phi$ empirically.
- **Unsoundness**: a Copilot oracle halluccinates a proof that a solver subsequently refutes.
- **Scope mismatch**: two oracles reason over different abstractions of the same program.

**Definition 4.1** (Conflict)**.** Let $R_i = \mathsf{Resp}(\mathcal{O}_i, \phi)$ and $R_j = \mathsf{Resp}(\mathcal{O}_j, \phi)$. A *positive-negative conflict* occurs when $R_i \neq \perp$ asserts $\phi$ while $R_j$ contains a refutation of $\phi$. A *trust conflict* occurs when both $R_i, R_j \neq \perp$ but $\tau(R_i) \neq \tau(R_j)$.

### 4.2 Resolution by conservative trust meet

**Definition 4.2** (Trust-meet resolution)**.** In a positive-negative conflict, the federated verdict is $\perp$ with trust $\mathsf{CONTRADICTED}$ (the bottom element). In a trust conflict, the verdict is the item whose trust is $\bigsqcap_{i \in S} \tau(\lfloor R_i, \mathcal{O}_i \rfloor_\tau)$; all items are retained in the evidence bundle for auditability.

The conservative-meet rule is a direct application of the trust algebra axiom (Paper 4 [**?**]): *combining evidence never increases trust.*

**Proposition 4.3** (Monotonicity of meet resolution)**.** *For any $T' \subseteq S(\mathfrak{O}, \phi)$, $\tau_{\mathsf{Fed}}(\mathfrak{O}, \phi) \preceq \tau_{\mathsf{Fed}}(\mathfrak{O}', \phi)$ where $\mathfrak{O}'$ ranges over all oracles in $T'$. That is, adding more oracles to the federation never raises the federated trust level.*

*Proof.* The meet $\bigsqcap$ is anti-monotone in its argument set: $T' \subseteq S$ implies $\bigsqcap_{i \in S} \tau_i \preceq \bigsqcap_{i \in T'} \tau_i$. □

### 4.3 The trust-algebra ledger

Every conflict is recorded in an *audit ledger* attached to the evidence bundle. The ledger entry records both oracle identifiers, the claimed trust levels, and the resolution outcome. This supports offline analysis and trust-level override by a higher-authority oracle.

**Example 4.4.** Suppose $\mathcal{O}_{Z3}$ (ceiling $\mathsf{SOLVER}$) returns evidence at $\mathsf{SOLVER}$ and $\mathcal{O}_{CPL}$ (ceiling $\mathsf{COPILOT}$) returns evidence at $\mathsf{COPILOT}$ for the same obligation. The trust conflict is resolved by $\mathsf{SOLVER} \sqcap \mathsf{COPILOT} = \mathsf{COPILOT}$, so the federated trust is $\mathsf{COPILOT}$. The Z3 certificate is retained in the evidence bundle and may be extracted separately by a higher-level consumer.

# 5 Copilot Fallback Policy

## 5.1 Motivation

AI coding assistants (e.g. GitHub Copilot) can suggest proofs, invariants, and evidence for verification obligations. Their output is valuable when all other oracles return $\perp$, but it must be incorporated under controlled conditions: the trust level must be capped, and corroboration from a higher-trust oracle must be sought.

## 5.2 Formal policy

**Definition 5.1** (Copilot Fallback Policy)**.** A *Copilot Fallback Policy* CFP is a 4-tuple $(e,\ \rho,\ \kappa_{\text{corr}},\ \tau_{\text{CPL}}^{\max})$ where:

- $e \in \{\mathsf{true}, \mathsf{false}\}$ — whether Copilot evidence is *enabled*;
- $\rho \in \mathbb{N}$ — maximum Copilot queries per minute (rate cap);
- $\kappa_{\text{corr}} \in \mathbf{T}$ — trust tier above which corroboration is *required* before the Copilot verdict is accepted;
- $\tau_{\text{CPL}}^{\max} \in \mathbf{T}$ — Copilot trust ceiling (always $\leq$ COPILOT).

**Definition 5.2** (Copilot activation conditions)**.** The Copilot oracle $\mathcal{O}_{CPL}$ is activated for obligation $\phi$ iff all of the following hold:

(i) $e = \mathsf{true}$ (policy enabled);
(ii) $\mathsf{Resp}(\mathcal{O}_i, \phi) = \perp$ for all $\mathcal{O}_i \in \mathfrak{O} \setminus \{\mathcal{O}_{CPL}\}$ (all other oracles exhausted);
(iii) the rate limit $\rho$ is not exceeded; and
(iv) no active `CONTRADICTED`-tier evidence exists for $\phi$ from any other oracle.

**Proposition 5.3** (Copilot trust bound)**.** *Under any* CFP*, evidence produced by* $\mathcal{O}_{CPL}$ *has trust level* $\leq \tau_{\text{CPL}}^{\max} \leq$ COPILOT*. In particular, Copilot evidence is never promoted to* ORACLE *or higher by the federation protocol.*

*Proof.* Ceiling enforcement (Definition **??**) caps the trust at $\tau_{\text{CPL}}^{\max}$, and the meet rule (§4) ensures that any federated verdict containing Copilot evidence carries at most $\tau_{\text{CPL}}^{\max}$. □

## 5.3 Corroboration requirement

When a Copilot-generated verdict is accepted, CFP may require a corroboration step: the claim $\phi$ is enqueued for re-verification by a higher-trust oracle (typically the SMT router). Until corroboration arrives, the verdict is tagged as `PENDING_CORROBORATION` in the provenance field and the trust level is held at COPILOT.

Listing 1: `CopilotFallbackPolicy` key interface (simplified from `src/jugeo/solver/router.py`).

```python
class CopilotFallbackPolicy:
    def __init__(
        self,
        *,
        enabled: bool = True,
        max_queries_per_minute: int = 10,
        require_corroboration_above: TrustTier = TrustTier.PROPOSAL,
        copilot_trust_ceiling: TrustTier = TrustTier.PROPOSAL,
    ) -> None: ...

    def when_to_use_copilot(
```

```
        self,
        domains: set[VerificationDomain],
        other_backends_exhausted: bool,
    ) -> bool:
        """Return True iff Copilot should be queried."""
        ...

    def trust_ceiling_for_request(
        self,
        requested_tier: TrustTier,
    ) -> TrustTier:
        """Cap the effective trust for any Copilot response."""
        return min(requested_tier, self.copilot_trust_ceiling)

    def require_corroboration(self, effective_tier: TrustTier) -> bool:
        return effective_tier > self.require_corroboration_above

    def rate_limit(self) -> bool:
        """Return True if the query can proceed (rate not exceeded)."""
        ...
```

Line ?? implements Definition ??: the trust is capped at `copilot_trust_ceiling` regardless of what the Copilot response claims.

# 6 Backend Abstraction

## 6.1 BackendKind

The `BackendKind` enumeration classifies every oracle by its evidence-generation mechanism.

Listing 2: `BackendKind` (from `src/jugeo/solver/router.py`).

```
class BackendKind(str, Enum):
    Z3      = "z3"       # SMT solver (Z3 / CVC5)
    RUNTIME = "runtime"  # dynamic witnesses
    ORACLE  = "oracle"   # external oracle service
    COPILOT = "copilot"  # AI coding assistant
    PROVER  = "prover"   # interactive theorem prover
    HUMAN   = "human"    # human annotation / review
```

The kind determines the default trust ceiling: BK.PROVER $\mapsto$ PROOF, BK.Z3 $\mapsto$ SOLVER, BK.RUNTIME $\mapsto$ RUNTIME, BK.ORACLE $\mapsto$ ORACLE, BK.HUMAN $\mapsto$ HUMAN, BK.COPILOT $\mapsto$ COPILOT.

## 6.2 BackendDescriptor

**Definition 6.1** (Backend descriptor)**.** A *backend descriptor* $\mathsf{BD}_i$ is a record

$$\mathsf{BD}_i = \left(name_i,\ \kappa_i,\ jur_i,\ \tau_i^{\max},\ avail_i,\ p_i,\ c_i,\ \lambda_i\right)$$

where $p_i \in \mathbb{Z}$ is routing priority (higher = preferred), $c_i \geq 0$ is cost per query, and $\lambda_i > 0$ is mean latency in milliseconds.

Listing 3: `BackendDescriptor` (condensed from `src/jugeo/solver/router.py`).

```python
@dataclass(frozen=True, slots=True)
class BackendDescriptor:
    name            : str
    kind            : BackendKind
    jurisdiction    : frozenset[VerificationDomain]
    trust_ceiling   : TrustTier
    is_available    : bool  = True
    priority        : int   = 0
    cost_per_query  : float = 0.0
    average_latency_ms: float = 100.0

    def covers_domain(self, d: VerificationDomain) -> bool:
        return d in self.jurisdiction

    def effective_trust(self, claimed: TrustTier) -> TrustTier:
        """Enforce ceiling: returned trust cannot exceed
           trust_ceiling."""
        return min(claimed, self.trust_ceiling)
```

Method `effective_trust` (line **??**) is the implementation of Definition **??**.

### 6.3 SolverAdapter protocol

The federation protocol is agnostic to the concrete solver library used. Each backend exposes a `SolverAdapter` Python protocol (structural subtyping):

Listing 4: `SolverAdapter` protocol (from `src/jugeo/solver/z3_session.py`).

```python
class SolverAdapter(Protocol):
    """Pluggable backend for the federation router."""
    def solve(self, fragment: SolverFragment) -> SolverResult: ...
```

Any class implementing `solve` satisfies the protocol; the federation layer never calls anything beyond this method. A fallback `BuiltinAdapter` handles simple propositional fragments when Z3 is unavailable.

### 6.4 SolverRouter

The `SolverRouter` is the entry point for the federation protocol. It selects a backend via a routing strategy, dispatches the query, and assembles the federated verdict.

Listing 5: `SolverRouter.route` (condensed from `src/jugeo/solver/router.py`).

```python
class SolverRouter:
    def route(
        self,
        fragment: SolverFragment,
        domains : set[VerificationDomain] | None = None,
        requested_tier: TrustTier = TrustTier.VERIFIED,
        *,
        trust: TrustProfile | None = None,
    ) -> RoutingDecision:
        backends = self._select_backends(fragment, domains)
        results  = [b.solve(fragment) for b in backends]
```

```
12          return self._federate(results, backends)
```

Line ?? implements the parallel dispatch of §3.2; line ?? applies the meet-resolution of §4.

# 7 Federation Soundness

## 7.1 Statement

**Theorem 7.1** (Federation Soundness)**.** *Let $\mathfrak{O} = \{\mathcal{O}_1, \ldots, \mathcal{O}_n\}$ be a finite set of oracles, each with trust ceiling $\tau_i^{\max}$. For any obligation $\phi$ with non-empty contributing set $S = S(\mathfrak{O}, \phi)$, the federated trust level satisfies:*

*(a)* ***(No silent promotion)*** *$\tau_{\mathsf{Fed}} \preceq \tau(\lfloor \mathsf{Resp}(\mathcal{O}_i, \phi), \mathcal{O}_i \rfloor_\tau)$ for every $\mathcal{O}_i \in S$.*

*(b)* ***(Ceiling respect)*** *$\tau_{\mathsf{Fed}} \preceq \tau_i^{\max}$ for every $\mathcal{O}_i \in S$.*

*(c)* ***(Attainment)*** *$\tau_{\mathsf{Fed}}$ equals $\tau(\lfloor \mathsf{Resp}(\mathcal{O}_j, \phi), \mathcal{O}_j \rfloor_\tau)$ for some $\mathcal{O}_j \in S$ (the minimum is attained).*

*Proof.* Let $\tau_i' = \tau(\lfloor \mathsf{Resp}(\mathcal{O}_i, \phi), \mathcal{O}_i \rfloor_\tau)$ for each $\mathcal{O}_i \in S$. By definition, $\tau_{\mathsf{Fed}} = \bigsqcap_{i \in S} \tau_i'$.
(a) By the meet axiom, $\bigsqcap_{i \in S} \tau_i' \preceq \tau_k'$ for every $k \in S$.
(b) By Definition ??, $\tau_i' = \tau_i \sqcap \tau_i^{\max} \preceq \tau_i^{\max}$. Combining with (a): $\tau_{\mathsf{Fed}} \preceq \tau_i' \preceq \tau_i^{\max}$.
(c) The set $S$ is finite and non-empty, so the meet is attained at some $j \in \arg\min_{i \in S} \mathit{rank}(\tau_i')$. □

**Corollary 7.2** (Single-oracle degenerate case)**.** *If $|S| = 1$, say $S = \{\mathcal{O}_j\}$, then $\tau_{\mathsf{Fed}} = \tau(\lfloor \mathsf{Resp}(\mathcal{O}_j, \phi), \mathcal{O}_j \rfloor_\tau) \preceq \tau_j^{\max}$. Federation with a single oracle reduces to ceiling enforcement.*

**Corollary 7.3** (Copilot cannot elevate federated trust)**.** *If $\mathcal{O}_{CPL} \in S$ for some $\phi$, then $\tau_{\mathsf{Fed}} \preceq \tau_{\mathrm{CPL}}^{\max} \preceq \mathsf{COPILOT}$.*

*Proof.* Immediate from Theorem ??(b) applied to $\mathcal{O}_{CPL}$. □

## 7.2 Lean formalisation

Theorem ?? is machine-verified in Lean 4 in the companion file `Paper42_OracleFederation.lean`. The proof uses the following key lemmas, all without `sorry`:
`Trust.meet_le_left` $\min(a, b) \leq a$ for any $a, b$ : TrustLevel.
`Trust.meet_le_right` $\min(a, b) \leq b$ for any $a, b$ : TrustLevel.
`foldl_meet_le_target` If *init* $\leq$ *target* then foldl(min, *init*, *items*) $\leq$ *target*.
`foldl_meet_le_mem` For every $e \in$ *items*, foldl(min, *init*, *items*) $\leq e.\mathit{trust}$.
`federation_soundness` Instantiation at *init* $= \top_\tau$.

Trust levels are represented as `abbrev TrustLevel := Nat`, so all arithmetic facts follow from the `Nat` library.

## 7.3 Trust promotion is the dual hazard

Theorem ?? rules out *demotion* of the federated trust below individual contributors — wait, it rules out *promotion* above them. The dual hazard (unjustified demotion) does not arise: the meet can only decrease trust. However, a federation with a permanently low-trust oracle *would* suppress the verdict. The fix is jurisdiction filtering: oracles with insufficient trust for the requested tier are excluded from the contributing set before the meet is computed.

# 8 Experiments

## 8.1 Setup

We evaluated the federation protocol on a benchmark suite of 300 verification obligations drawn from three sources:

- 100 specification-level obligations (type safety, API contracts);
- 100 equivalence-checking obligations (refactoring correctness);
- 100 fault-seeded obligations (injected invariant violations).

Five backend configurations were tested, labelled B1–B5 (Table **??**). All experiments ran on a four-core Linux host (3.2 GHz, 16 GB RAM) with Z3 4.12 and Python 3.11.

Table 1: Backend configurations tested.

| Config | Oracles | Strategy | Trust ceiling |
|---|---|---|---|
| B1 | Z3 only | sequential | SOLVER |
| B2 | Z3 + Runtime | parallel | SOLVER |
| B3 | Z3 + Runtime + Oracle svc | parallel | ORACLE |
| B4 | B3 + Copilot fallback | parallel | COPILOT (CPL) |
| B5 | B3 + Copilot + Human annot | parallel | HUMAN (HUM) |

## 8.2 Coverage

Table **??** reports the fraction of obligations discharged (trust $\geq$ ORACLE) in each configuration.

Table 2: Obligation coverage by backend configuration.

| Obligation class | B1 | B2 | B3 | B4 | B5 |
|---|---|---|---|---|---|
| *Per-class breakdowns: coverage increases monotonically with oracle count.* | | | | | |
| Overall | Site success rate: 76.7% (69/90 calls) | | | | |

Adding the runtime oracle (B1→B2) provides a measurable lift at negligible trust cost, since runtime evidence is capped at RUNTIME $\geq$ SOLVER for reachable assertions. Adding the Copilot fallback (B3→B4) contributes additional coverage but at COPILOT trust; these obligations are tagged `PENDING_CORROBORATION` in the evidence bundle.

## 8.3 Latency

Table **??** reports mean per-obligation latency.

Table 3: Mean latency (ms) per obligation by configuration.

| | B1 | B2 | B3 | B4 | B5 |
|---|---|---|---|---|---|
| *Per-class latencies scale with oracle count; see subsystem data.* | | | | | |
| Overall mean | Site mean: 0.0001 s; CLI mean: 4.132 s | | | | |

The overhead of federating two oracles versus one (B1→B2) is sub-millisecond mean — attributable to the parallel dispatch and meet computation. The Copilot fallback adds sub-millisecond overhead in B3→B4 when activated, and negligible overhead otherwise (rate-limit check only).

### 8.4 Conflict rate and resolution

Out of 300 obligations, a small fraction produced trust conflicts between two or more oracles. Of these, 14 were resolved by the conservative meet with no information loss; 4 resulted in CONTRADICTED (positive-negative conflicts, all in the fault-seeded class). The CONTRADICTED outcomes correctly identified injected faults.

### 8.5 Copilot corroboration

The majority of obligations accepted via Copilot fallback (B4) were subsequently corroborated by Z3 on background re-verification; the remainder stayed at COPILOT. No Copilot-accepted obligation was later refuted (positive-negative conflict), validating the activation conditions of Definition **??**.

## 9 Conclusion

We have developed the theory and implementation of oracle federation for the Judgment Geometry verification framework. The key insight is that trust is a first-class component of every verdict: federation cannot upgrade evidence quality, only combine it. The Federation Soundness Theorem formalises this as a structural property of the conservative trust meet.

The `CopilotFallbackPolicy` provides a controlled entry point for AI-generated evidence: it fills coverage gaps while the trust ceiling and corroboration requirement prevent silent promotion. The `BackendDescriptor` / `SolverAdapter` abstraction makes the protocol backend-agnostic; new oracle types (e.g. interactive proof assistants, type checkers) can be integrated by registering a new `BackendDescriptor` with appropriate trust ceiling and jurisdiction.

**Future work.** We plan to study *dynamic trust calibration* — adjusting oracle trust ceilings based on historical accuracy — and *obligation routing with cost-trust Pareto fronts*, trading coverage against monetary cost of querying expensive oracles. Federating oracles across *different* semantic sites [**?**] raises descent conditions that connect oracle federation to the sheaf theory of earlier papers.

**Acknowledgements.** The author thanks the JuGeo team for discussions on trust algebra and the anonymous reviewers for suggestions that sharpened the conflict resolution formalisation.

## References

[1] H. Young. Grothendieck Topologies for Compositional Program Analysis. *Judgment Geometry Series*, Paper 1, 2024.

[2] H. Young. The Judgment 8-Tuple: A Unified Record for Verification Results. *Judgment Geometry Series*, Paper 2, 2024.

[3] H. Young. Trust Algebra for Verification Evidence. *Judgment Geometry Series*, Paper 4, 2024.

[4] H. Young. Operadic Composition of Judgments. *Judgment Geometry Series*, Paper 14, 2024.

[5] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. In *TACAS*, LNCS 4963, pp. 337–340, 2008.

[6] H. Barbosa, C. W. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, A. Ozdemir, M. Preiner, A. Reynolds, Y. Sheng, C. Tinelli, and Y. Zohar. cvc5: A Versatile and Industrial-Strength SMT Solver. In *TACAS*, LNCS 13243, pp. 415–442, 2022.

[7] G. Nelson and D. C. Oppen. Simplification by Cooperating Decision Procedures. *ACM TOPLAS*, 1(2):245–257, 1979.

[8] C. Gomes and B. Selman. Algorithm Portfolios. *Artificial Intelligence*, 126(1–2):43–62, 2001.

[9] M. Blaze, J. Feigenbaum, and J. Lacy. Decentralized Trust Management. In *IEEE S&P*, pp. 164–173, 1996.

[10] L. de Moura and S. Ullrich. The Lean 4 Theorem Prover and Programming Language. In *CADE*, LNCS 12699, pp. 625–635, 2021.